REPLACING REPOSITORY EXPLORATION WITH TYPED JEV DECISIONS
CS329Z MIDWAY REPORT DRAFT

Kristjan Dagur Egilsson **Ómar Ingi Halldórsson** **Rami Ratl Mrad**

1 RESEARCH QUESTIONS

Our project asks whether a structured decision model can replace a narrow component of an existing coding agent at lower total cost while preserving bug-repair performance. Following the instructor's recommendation to obtain mechanical evidence before redesigning an agent harness, we study OpenCode's repository-exploration subagent. Our intervention uses Jev to select relevant source files from deterministic search candidates; the same strong generative model continues to write and test patches. We compare this intervention against the native generative explorer and a search-only replacement. The primary outcomes are independently graded repair success and total inference cost.

We report two experiments. A development pilot on five SWE-bench Lite issues fixed the method. A second pilot then applied the unchanged method to ten SWE-bench Verified tasks, selected before any agent run and graded by the official containerized harness. The midpoint objective is a working substitution and an authentic evaluation pipeline, not a claim of calibrated confidence or performance equivalence from small samples.

2 ENVIRONMENT & DATA

Existing agent harness. We use the pinned OpenCode 1.19.5 command-line harness and retain its native build agent, `task` delegation tool, `explore` subagent, file tools, and patch-writing loop (OpenCode contributors, 2026). Each rollout starts from an exact upstream repository snapshot without upstream git history. The issue prompt asks the build agent to delegate initial localization to the existing explorer before editing, then implement a minimal fix and optionally run existing tests. The prompt is identical across conditions within an experiment. The parent agent retains its own read and search tools and can therefore recover from imperfect localization. Tool permissions allow edits only to the repository's implementation directories, never to tests; these permissions are not an operating-system sandbox.

Development pilot (SWE-bench Lite). We used all five pydicom issues in the official SWE-bench Lite development split (Jimenez et al., 2024; SWE-bench contributors, 2026a), chosen for a locally runnable pure-Python environment. Task IDs and base commits were frozen before agent outcomes. Each patch was evaluated by applying it and the benchmark's test patch to a fresh snapshot and running the published fail-to-pass and pass-to-pass tests under pinned local pytest. Every environment was validated beforehand: the broken base fails and the upstream fix passes. This local evaluation is not an official SWE-bench score.

Verified pilot sample. SWE-bench Verified is a 500-task subset of the SWE-bench test split whose issues and tests were screened by human annotators for adequate specification (OpenAI, 2024). We pinned dataset revision `78f471b` (SWE-bench contributors, 2026b) and drew a 50-task sample, stratified by repository in proportion to the full set; within each repository, tasks are ordered by a SHA-256 hash of a recorded seed and the instance ID. The first pilot uses a nested 10-task subset produced by the same rule: five Django tasks, two SymPy, and one each from Matplotlib, scikit-learn, and Sphinx. Selection used only instance IDs and repository names. Both the sample and the protocol were committed before any agent run.

Official evaluation and environment checks. Verified patches are graded by the official `swebench` 5.0.2 harness (Jimenez et al., 2024) using the published per-task Docker images and a local copy of the pinned rows. Before any paid run, we checked every image. Each contains the base commit plus one harness setup commit and no later upstream history. That setup commit changes no source or test content; it only alters file permissions, except for dependency pins in Sphinx's packaging files. Neither the benchmark's test patch nor the gold fix is present in any image, and all ten gold fixes resolve under the harness. Public issues may appear in model training data, so we make no contamination-free claim.

3 METHODS

One internal substitution. The build agent uses `openai/gpt-6.1-sol` in every condition. A thin OpenAI-compatible provider adapter exposes distinct build and explore URLs. OpenCode's native `task` tool still creates an explorer session and returns its result to the parent. In the native condition, both URLs forward to the same strong model, so the explorer is a GPT-6.1-Sol subagent with read-only tools. In the Jev condition, only explore-provider requests are replaced by deterministic retrieval, one typed Jev request, and a formatted list of selected file paths and snippets.

Candidate construction and Jev decisions. We index the repository's Python implementation files, excluding tests. A deterministic BM25-style search ranks files against the parent agent's actual exploration request. The top 12 candidates include relative paths and the most query-matching 16-line window of each file, capped at 1,800 characters. Jev 1.14 receives the request and candidates and answers one Noul file-relevance question per candidate in a single API call (OpenRouter, 2026). We validate every returned probability and return the three highest-scoring files to the parent. Jev neither writes patches nor selects the coding model. An API error or invalid score falls back to the native explorer, and both costs are counted. No confidence threshold is applied, and relevance scores are not calibrated probabilities of repair success.

Comparisons and fixed limits. Each experiment runs three conditions on the same tasks, one rollout each: native exploration, retrieval plus Jev selection, and search-only, which returns the top three retrieval candidates without Jev. The search-only control tests whether any benefit requires the decision model. All conditions share the writer model, tools, evaluator, 16 parent steps, 6 explorer steps, at most 32 requests, 4,096 output tokens per request, and 480 seconds per run. The Verified protocol imports these settings from the development protocol and refuses to run if any of them differ.

Agent-side testing on Verified. Verified repositories require version-specific environments, so the agent's test command runs inside the task's official image. Each run starts one container, completes the image's environment setup online, then disconnects the network and replaces the repository's git history with a single fresh commit. The agent may call `run_tests` with test targets. The wrapper applies the current workspace diff to a clean copy and invokes the repository's standard test runner with the validated, shell-quoted targets. The benchmark's own test targets, test patch, and gold patch never reach the agent.

Accounting and artifacts. The adapter records upstream responses and billed usage for writer, auxiliary, exploration, and Jev calls. A durable ledger reserves conservative costs before requests and settles bills; caps are $2 per rollout and $10 for the Verified pilot. The real API key stays in the host adapter. Each run stores its source/protocol hash, task metadata, event trace, request/response receipts, file decisions, test-command log, patch, and grader report. All 30 Verified pilot runs share one source hash, and every bill was captured.

4 SUMMARY OF PROGRESS

Measured comparison. Table 1 summarizes both pilots. On the five development issues, the native and Jev conditions each solved 2/5, with mean cost falling from $0.110382 to $0.072762 (34.1%). On the ten Verified tasks, native and Jev each resolved 9/10 and search-only resolved 8/10. Mean total cost fell from $0.106 for native to $0.056 for Jev, a 48% reduction, and mean agent time

Table 1: Both pilots, one rollout per task and condition; cost includes all captured model calls; time is agent wall time excluding grading. Top: five pydicom Lite development issues, local regression tests. Bottom: ten SWE-bench Verified tasks, official Docker harness, with mean cost split into the explore step and the parent writer.

Condition	Solved	Cost (USD)	Explore	Writer	Time (s)
Development pilot (pydicom, SWE-bench Lite dev)					
Native generative explorer	2/5	0.110			37.6
Jev file selection	2/5	0.073			25.9
Search only	2/5	0.062			24.9
Verified pilot (10 tasks, 5 repositories)					
Native generative explorer	9/10	0.106	0.0572	0.049	42.9
Jev file selection	9/10	0.056	0.0002	0.056	26.0
Search only	8/10	0.054	0.0000	0.054	25.3

fell from 42.9 to 26.0 seconds. The Jev runs used ten decision calls and no fallbacks. They replaced 48 explorer generation calls, which cost the native condition $0.057 and 21.7 seconds per task on average. There were no timeouts or forbidden edits. The Verified pilot cost $2.17 OpenRouter.

Where the saving comes from. The search-only control matched Jev's cost and came within one task of its success, so the experiment does not show that Jev is preferable to deterministic retrieval. Its single additional failure (Django 15695) is not a retrieval failure: search returned the file containing the upstream fix, and the writer edited it but placed its guard after an index lookup that fails. To examine localization separately from success, we compared the returned files with the files changed by the upstream fix. This analysis uses evaluator-side data that never reached the agent. The fix file appeared among the three returned files in 7/10 Jev runs and 6/10 search-only runs. Nonetheless, the writer edited that file in all 30 Verified runs. In the traces we inspected where the returned files missed the fix file, the parent issued its own searches immediately after delegating exploration and located the file itself. The saving therefore comes primarily from not running a generative exploration subagent whose work the parent largely repeats, rather than from the quality of Jev's selections. Jev's highest relevance score also did not separate correct from incorrect selections in this sample: misses scored up to 0.85, and hits as low as 0.66. All three conditions failed SymPy 13852, and the development pilot's failures likewise reflected incomplete repairs, not infrastructure errors.

Completed work and next steps. We have completed the development pilot, a Linux-validated reproduction, the Verified pipeline with environment checks, and the 30-run Verified pilot. The pipeline has 27 offline checks covering method equivalence with the development protocol, leakage boundaries, test-command validation, and grading edge cases. Next, we will add an empty-explorer control in which the explore call returns no files. If it matches Jev and search, exploration is not the decision through which Jev can matter for this writer. We will then extend to the remaining 40 sampled tasks (about $9 projected from measured costs) and report paired uncertainty. We also plan to move Jev to a decision that the agent cannot bypass: uncertainty-gated escalation between a cheap and a strong writer, with thresholds chosen on independent development data. The original BANKING77/CLINC150 intent-classification track remains deferred. Training with Tinker credits is conditional on a demonstrated need.

Safety considerations. We use public software issues and repository fixtures, not clinical data or deployed systems. Generated patches are research artifacts and are never merged upstream. Credentials are excluded from traces and from agent processes. Agent-side tests run in network-disabled containers, but the agent's file edits are restricted only by tool permissions on the host.

REFERENCES

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *International Conference on Learning Representations*. https://arxiv.org/abs/2310.06770.

SWE-bench contributors. SWE-bench Lite dataset, development split. Revision 6ec7bb89b9342f664a54a6e0a6ea6501d3437cc2. Accessed November 6, 2026. https://huggingface.co/datasets/princeton-nlp/SWE-bench_Lite.

SWE-bench contributors. SWE-bench Verified dataset, test split. Revision 78f471bf655a3137b2e8a75af1501690ec009ec3. Accessed November 6, 2026. https://huggingface.co/datasets/SWE-bench/SWE-bench_Verified.

OpenAI. 2024. Introducing SWE-bench Verified. https://openai.com/index/introducing-swe-bench-verified/.

OpenCode contributors. OpenCode 1.19.5, native task/explore implementation. Software source. Accessed November 6, 2026. https://github.com/anomalyco/opencode/blob/v1.19.5/packages/opencode/src/tool/task.ts.

OpenRouter. Jev documentation: using the TypeSafe decision model on OpenRouter. API documentation. Accessed November 6, 2026. https://openrouter.ai/docs/guides/community/jev.

AI-USE DISCLOSURE STATEMENT

OpenAI Codex assisted Rami with interpreting the instructor feedback, inspecting source and documentation, implementing the provider adapter and local evaluator, executing the development pilot, and drafting the first version of this report. Anthropic's Claude (Claude Code, Opus 5.5) assisted Kristjan with Linux reproduction, the API-key loading change, selecting the Verified sample, implementing and checking the Verified runner and analysis, interpreting the results, and revising this report. Jev 1.14 made file-relevance decisions within the evaluated agent; GPT-6.1-Sol produced the experiments' patches and native exploration responses. Benchmark issues, upstream fixes, and tests come from SWE-bench rather than being generated for this project. This draft requires team review and confirmation against the course disclosure policy.